RNArobo Search

Find *structural* RNA motifs (base-paired helices plus unpaired loops, not a flat nucleotide string) by describing the shape you want and letting the bundled RNArobo 2.1.0 engine scan your FASTA for it.

Overview

Most pattern search in RSAS is linear. The Motif / Sequence Finder takes an IUPAC string like `AGGAGG` or `NNUANN` and reports every place that exact run of bases appears. That is the right tool when the thing you are hunting for is a sequence.

RNArobo Search is for the other case: when the motif is defined by its *structure*. A stem-loop, a three-way junction, an RNA thermometer hairpin, a pseudoknot. These are characterized by which bases pair with which (the helices) and which stay unpaired (the loops), and the actual nucleotides can vary a lot between instances. You cannot capture "a stem of any 5 base pairs closing a 6 nucleotide loop" with a flat IUPAC pattern, because the two strands of the stem have to be reverse-complementary to each other, and a linear pattern has no way to express that relationship.

RNArobo does. You write a *descriptor*: an ordered list of elements (helices, single-stranded regions, relational stems) plus a *motif map* that lays them out 5' to 3'. RSAS builds the descriptor file, shells out to the `rnarobo` binary, and parses the matches back so you can review and export them. The search itself is RNArobo's; RSAS is the interface around it.

The engine is RNArobo (Rampášek et al. 2016, *BMC Bioinformatics* 17:216, doi:10.1186/s12859-016-1074-x), bundled at version 2.1.0.

When to use it

Use RNArobo Search when the pattern you care about is defined by base pairing rather than by a fixed string of bases:

- A **stem-loop / hairpin** where the loop sequence varies but the stem has to fold back on itself.
- A **multi-stem motif** like a two-stem junction or a three-stem RNA thermometer, where several helices and loops sit in a defined order.
- Any motif where you want to allow a few **mismatches or mispairs** in the stem, or short **insertions** in a loop, and still call it a match.
- Cases where you need to control the pairing rules, for example forcing **canonical Watson-Crick only** versus **allowing G-U wobble**.

Reach for the linear Motif / Sequence Finder instead when the target really is a sequence (a Shine-Dalgarno box, a start codon, a known consensus motif with no structural constraint). That tool is

simpler, faster to set up, and integrates into the per-sequence analysis columns. RNArobo Search is a standalone search that produces its own match list and TSV export.

Step by step

1. **Open the page / dialog.** The status line at the top tells you whether the `rnarobo` binary was found. A green check with a path means you are ready; a red cross means the binary is missing (see Troubleshooting).

2. **Start from a preset or a motif map.** The fastest scaffold is the **Motif Map** field. Type the layout of your motif as space-separated element names, for example `h1 s1 h1'` for a simple hairpin, then click **Auto-fill Elements**. RSAS reads the map, works out each element's type from the first letter of its name (`h` helix, `s` single-stranded, `r` relational), and creates one empty row per distinct element. Alternatively pick an entry from the **Preset** dropdown and click **Load Preset** to fill in both the map and fully tuned element rows.

3. **Tune the elements.** Each element gets a row (or pair of rows) where you set its mismatches, mispairs, insertions, and its sequence pattern(s). For a helix you fill in a 5' strand and a 3' strand; for a single-stranded element you fill in one sequence; for a relational element you also give a transform string. Fields that do not apply to the current type are hidden or disabled, so a single-stranded row will not show you a mispairs box. The **Descriptor Preview** updates live so you can see the exact descriptor text RSAS will hand to the engine.

4. **Point at a FASTA.** Set the **Sequence File** with **Browse** (or type a path). Sequences loaded on the Analyze page carry over as the starting point.

5. **Set search options and run.** Decide on both-strands, non-overlapping, and an N-ratio threshold (all covered below), then click **Run Search**. The search runs on a background thread and streams progress into the Results box. When it finishes you get a table of matches, and **Export Results** lights up so you can save them as TSV.

Options in detail

Element types and their fields

Every element has a **name**, a **type**, and per-element tolerance fields (**mismatches**, **mispairs**, **insertions**, **insertion nucleotide**). The name's first character is load-bearing: RNArobo decides an element's type from the first letter of its name, so a helix must be named `h...`, a single-stranded region `s...`, and a relational stem `r...`. RSAS validates this and refuses to run a descriptor where the name prefix disagrees with the chosen type, because RNArobo would otherwise silently treat (say) a helix named `stem1` as single-stranded and give you scientifically wrong matches with no error.

Helix (`h`) is a paired stem. It has two strands, a **5' strand** and a **3' strand**, which fold back and pair with each other. In the descriptor a helix is written `h1 mismatches:mispairs pos_strand:neg_strand`. By default a helix allows G-U wobble pairing. Use `*` as a wildcard inside a strand pattern (the preset strands like `NNN**CC` / `GG**NNN` show the style), and `N` for any single base.

Single-stranded (`s`) is an unpaired region: a loop, a bulge, a linker. It has a single **sequence** pattern, written `s1 mismatches sequence`. The sequence can be a concrete IUPAC pattern (`ACCRNNT`), all-`N` for "any bases of this length", or use `*` to match a variable-length stretch. One important constraint: a `*` wildcard must sit at the start or the end of the sequence, not flanked by nucleotides on both sides. A pattern like `N*N` causes RNArobo to backtrack exponentially and the search hangs, so RSAS rejects it up front and tells you to use `NNN*`, `*NNN`, or `*` alone.

Relational (`r`) behaves like a helix (two strands, mismatches, mispairs) but adds a **transform** string that defines the pairing rules. The transform lists what pairs with A, C, G, T in that order. `TGCA` restricts the stem to canonical Watson-Crick pairs only; `TGYR` reproduces the default helix behavior (Watson-Crick plus G-U wobble). Only relational elements carry a transform string; a plain helix already permits wobble through RNArobo's built-in behavior. Reach for a relational element when you need pairing rules different from the default helix, the most common being "canonical only, no wobble". In the descriptor it is written `r1 mismatches:mispairs pos_strand:neg_strand transform`.

Mismatches, mispairs, insertions

These are the three tolerance knobs, all defaulting to 0 (an exact match):

- **Mismatches** apply to all element types. They allow that many positions in the element's sequence pattern to differ from the target. Loosening this finds motifs whose loop or strand sequence drifts from your pattern.
- **Mispairs** apply only to helices and relational elements. They allow that many positions in the stem to be unpaired (a non-pairing base across the helix) and still call the helix a match. This is how you tolerate a small internal bulge or a single broken pair in an otherwise good stem.
- **Insertions** apply to any element. They allow up to that many single-nucleotide insertions relative to the pattern, and you can constrain what may be inserted with the **insertion nucleotide** (an IUPAC code, for example `A` for adenine only or `Y` for any pyrimidine). With insertions on, the descriptor specs grow a third field: a helix becomes `mismatches:mispairs:insertions` and `pos:neg:insert_nuc`, and a single-stranded element becomes `mismatches:insertions` and `sequence:insert_nuc`.

All three must be non-negative; RSAS validates that before running.

Motif map shorthand and primed names

The **motif map** is the spine of the descriptor: a space-separated list of element names in 5'-to-3' order. It does two jobs. It tells RNArobo the layout (which element comes after which), and through **primed names** it tells the engine which strands belong to the same helix.

A helix occupies two positions in the map, written `h1` for its 5' strand and `h1'` (primed) for its 3' strand, the complementary strand that pairs back with it. So `h1 s1 h1'` reads as "a stem, then a loop, then the stem's other side closing back", a hairpin. A two-stem junction like `h1 s1 h2 s2 h2' s3 h1'` nests one helix inside another. Only helix and relational elements take a primed form; a single-stranded element pairs with nothing, so RSAS rejects a primed `s` name. Every name in the map must also have a matching element specification, or the descriptor is incomplete.

Auto-fill uses the same shorthand in reverse: it splits the map on spaces, strips the prime off each token, and creates one element row per distinct base name, choosing the type from the first letter.

Search options

- **Both strands** (`-c`, on by default). Also searches the reverse complement of each sequence, so a motif on the minus strand is found. Matches found on the reverse strand are reported with a `-` strand marker. Leave this on unless you specifically only want the given orientation.
- **Non-overlapping** (`-u`, off by default). Reports only disjoint matches. With it off (the default) RNArobo can report overlapping hits in the same region; with it on you get a non-overlapping subset, which is cleaner when you only care about whether and roughly where the motif occurs.
- **N-ratio threshold** (optional, blank by default). A number between 0.0 and 1.0 passed to the engine as `--nratio`. It is the maximum fraction of `N` (ambiguous) bases a sequence may contain before it is skipped. Useful for filtering out low-quality or heavily masked sequences. Leave it blank to apply no threshold. RSAS validates that what you type is a number in `[0.0, 1.0]`.

(FASTA output, RNArobo's `-f` flag, is deliberately not exposed in the UI, because the results parser only understands RNArobo's default tabular output.)

What you get

When the search finishes, the **Results** box shows a table, one row per match, with:

- **Seq Name**, the FASTA record the match landed in.
- **From** and **To**, the 1-based coordinates of the match within that sequence.
- **Strand**, `+` for the forward strand, `-` for a reverse-complement hit (only possible when both-strands is on).
- The **matched elements** line beneath each hit: RNArobo's pipe-delimited breakdown showing what each element of your descriptor matched.

At the bottom you get the **total match count** and the **total bases scanned**. If nothing matched, RSAS says so explicitly (and still reports bases scanned), which is your cue to loosen tolerances or re-check the descriptor.

TSV export

Click **Export Results** (enabled only once there is at least one match) to save the table. The file is tab-separated with this header:

```
Seq_Name    From    To  Strand  Description Matched_Elements
```

One row per match. The `Description` column carries any FASTA description text, and `Matched_Elements` carries RNArobo's per-element breakdown. Embedded tabs and newlines in any field are replaced with spaces so the TSV stays well-formed. The default filename is `rnarobo_results.tsv` and the default location is your Downloads folder.

How it works

The whole feature is a thin wrapper around the `rnarobo` command-line binary; the search logic lives in the engine, not in RSAS.

1. **Binary resolution.** `_get_rnarobo_binary()` (in `RnaThermofinder/utils/rnarobo_wrapper.py`) looks for the binary in three places, in order: inside a PyInstaller frozen bundle (`bin/rnarobo`), then in the project's `bin/<platform>/` directory (`macos`, `linux`, or `windows`), then anywhere on the system `PATH`. `check_rnarobo_available()` drives the green/red status line in the dialog and marks the binary executable if it is not already.

2. **Descriptor assembly.** The UI rows are read into `DescriptorSpec` objects (`HelixElement`, `SingleStrandedElement`, `RelationalElement`). `DescriptorSpec.to_descriptor_text()` and `_format_element()` turn those into the exact descriptor-file text shown in the preview, assembling the `mismatches:mispairs[:insertions]` and `pos:neg[:insert_nuc]` field structure per element type. `validate_descriptor()` runs first and catches the common mistakes: empty map, a map token with no spec, a primed single-stranded name, a name whose prefix disagrees with its type, negative tolerances, and the dangerous `N*N`-style flanked wildcard.

3. **Run.** `run_rnarobo()` writes the descriptor to a temporary `.desc` file, builds the command line (`-c` for both strands, `-u` for non-overlapping, `--nratio <value>` when set, then the descriptor path and the sequence file), and calls the binary with `subprocess.run(..., timeout=600)`. The temp descriptor file is always cleaned up afterward.

4. **Parse.** `_parse_rnarobo_output()` walks RNArobo's default tabular stdout: it skips the header block, reads each match as a position line plus its pipe-delimited element line, and pulls the total-bases-scanned figure out of the `----- SEARCH DONE -----` footer. The result comes back as an `RNAroboResult` (a list of `RNAroboMatch` plus totals and the raw stdout/stderr and return code).

The dialog runs the call on a background thread so the UI stays responsive, captures the option checkbox values in the main thread first (tkinter `BooleanVar` reads are not thread-safe), and reports a

non-zero exit code back to you even when the engine printed nothing to stderr, so a bad descriptor does not fail silently.

Worked example

A minimal stem-loop. The motif is "a short stem closing a small loop", and you do not care about the exact loop sequence.

1. In the **Motif Map** field, type:

   ```
   h1 s1 h1'
   ```

   This reads as: helix 5' strand, then a loop, then the helix 3' strand closing it back into a hairpin.

2. Click **Auto-fill Elements**. RSAS creates two rows: a helix `h1` and a single-stranded `s1`. (You do not get a third row for `h1'`, it is the same helix, just its other strand.)

3. Fill in `h1`. Set the **5' strand** to `NNNNNN` and the **3' strand** to `NNNNNN` for a 6 base-pair stem of any sequence, leave mismatches and mispairs at 0 for an exact stem, or set mismatches to 1 to tolerate one off base. The helix allows G-U wobble by default.

4. Fill in `s1`. Set its **sequence** to `NNNN` for a fixed 4 nucleotide loop, or use `NNNN*` if you want "at least four, then any length". Remember `*` must be at the end, not in the middle.

5. The **Descriptor Preview** now shows something like:

   ```
   h1 s1 h1'

   h1 0:0 NNNNNN:NNNNNN
   s1 0 NNNN
   ```

6. Point **Sequence File** at your FASTA, leave **Both strands** on, and click **Run Search**. Every place a 6 bp stem closes a 4 nt loop (on either strand) comes back as a match, with its coordinates.

To restrict the stem to canonical Watson-Crick pairs only (no G-U wobble), change `h1` to a **relational** element named `r1` (and the map to `r1 s1 r1'`) with the transform set to `TGCA`. The bundled "Relational (canonical only)" preset is exactly this.

Tips

- **Scaffold with the motif map, then tune.** Typing the layout and hitting Auto-fill is faster and less error-prone than adding element rows one at a time, because it guarantees the names and types

line up with the map.

- **Start strict, then loosen.** Run with 0 mismatches / 0 mispairs first. If you get too few hits, raise mismatches on the loop or mispairs on the stem one at a time and watch the count climb. It is easier to reason about than starting loose.
- **Use `N` for "any base, fixed length" and `*` for "any length".** They are different tools: `NNNN` is exactly four positions, `NNNN*` is four or more.
- **Watch the live preview.** It is the literal descriptor text the engine receives, so if a match count surprises you, read the preview before changing anything else.
- **Reach for a relational element only when you need non-default pairing.** For most stems the default helix (with wobble) is what you want; relational with `TGCA` is the "strict canonical" escape hatch.
- **Both strands doubles the search space.** If you know your motif's orientation and want a faster, cleaner result, turn it off.

Limitations and gotchas

- **The `rnarobo` binary ships for macOS only.** The repository bundles a macOS build at `bin/macos/rnarobo`. On Linux, download the ready-made executable published by RNArobo's authors (`https://compbio.fmph.uniba.sk/rnarobo/rnarobo-2.1.0-linux64`) into `bin/linux/rnarobo` of your RSAS folder, or build it from source; on Windows, use WSL2. No administrator rights are needed, and RSAS picks the binary up automatically on the next launch. Commands: INSTALL.md, section 8.
- **The name's first letter decides the type, inside RNArobo.** This is an RNArobo design, not an RSAS one. RSAS guards against it, but it is why you cannot name a helix `stem1`, the engine would read the leading `s` and treat it as single-stranded.
- **Wildcards flanked on both sides hang the engine.** `N*N` (or any `*` with nucleotides before and after it) triggers exponential backtracking in RNArobo. RSAS blocks this at validation time; keep `*` at the start or end of a single-stranded pattern.
- **The parser expects the default tabular output.** That is why RNArobo's FASTA-output flag (`-f`) is not exposed; turning it on would produce output the result parser cannot read.
- **Long or pathological searches.** The subprocess call has a 600-second timeout. A very large FASTA combined with a loose descriptor (many mismatches, broad wildcards) can be slow; tighten the descriptor or reduce the input if a search does not return.

Troubleshooting

The status line shows a red cross / "RNArobo not found". The binary could not be resolved from the frozen bundle, `bin/<platform>/`, or `PATH`. On macOS, confirm `bin/macos/rnarobo` exists and is executable. On Linux, download RNArobo into `bin/linux/` of your RSAS folder (see Limitations above

and INSTALL.md, section 8) and reopen the dialog. The **Run Search** button stays disabled until a binary is found.

The search exits with a non-zero status and no clear error. RSAS reports the exit code even when RNArobo prints nothing to stderr, and the usual cause is a descriptor the engine rejected. Re-read the **Descriptor Preview**: check that every element name in the motif map has a spec, that helix names start with `h` and relational with `r`, and that no single-stranded pattern has a `*` in the middle. The validator catches most of these before the run, but a malformed strand pattern can still make the engine unhappy.

No matches found. This is a real, valid result, not an error, the box also reports total bases scanned to confirm the search ran. Your descriptor is likely too strict. Raise mismatches (loops) or mispairs (stems), widen a fixed loop length, or switch a too-specific sequence pattern to `N`s. If you expected hits on the opposite strand, confirm **Both strands** is on.

Export Results is greyed out. It only enables after a search returns at least one match. Run a search that finds something first.